



南開大學
Nankai University

引言

2025年9月12日

成绩分布

● 平时项目作业：35%

● 出勤：5%

● 小组前端项目：60%

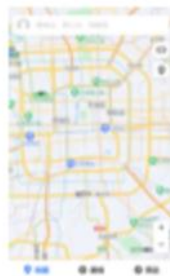
- 项目成果评价：项目功能完整性考核、项目代码质量考核
- 项目过程评价：项目进度把控、团队协作与沟通
- 项目展示评价：展示内容与质量、汇报表达与展示技巧

什么是Web前端

使用Web服务



信息服务



地图服务



音乐服务



视频服务



点餐服务

人机对话：人通过与前端交互来获得后端服务



Web前端不断发展，开发难度变大

早期网站主要内容都是静态的，以图片和文字为主，用户使用网站的行为也以浏览为主。
随着互联网技术的发展和HTML5、CSS3的应用，现代网页更加美观，交互效果更显著，功能更强大。

内容 (文本/图片/视频.....) + 超链接
Web网页



移动互联网发展带来了大量高性能的移动终端设备。HTML5, Node.js的广泛应用，各类UI框架，JS类库层出不穷，开发难度也在逐步提升。

前端开发是一个不断变化的领域，随着技术的进步，可以创造出更出色的用户体验，让产品更强大和吸引人。

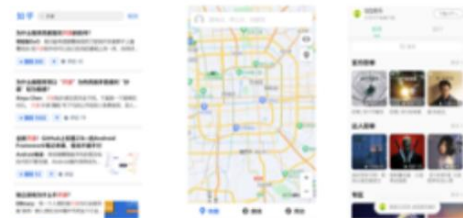


早期的网页



现代网页

排版、动画更美观



移动互联网

实时性、交互性、多媒体

为什么要学前端

前端适用场景



PC



移动App



小程序



游戏



服务端

- 掌握技能，获得就业机会
- 为提供的服务/软件功能实现人机交互界面
- 将设计转为应用
- 追求更好的体验，吸引更多的用户
-

前端开发核心技术与领域细分方向

核心技术包括但不限于：

- HTML：网页的核心，用于构建万维网页面的标准语言
- CSS：控制网页外观，对网页内容进行设计、布局或添加动画等，使网页更加美观
- JavaScript：动态客户端脚本语言，用于增强网页的交互性
- 性能优化：减少HTTP请求、避免重定向、优化DOM元素数量等
- 前端框架：如Bootstrap、Vue.js、React等，加速Web前端开发



前端领域细分方向：

- **低代码方向**：熟悉组件拖拽、流程管理，以及可视化框架如EChart.js和D3.js。这个方向专注于构建低代码应用。
- **前端可视化方向**：通过使用 ECharts.js、AntV、D3.js、Canvas、SVG、Three.js 等前端图表库，实现数据大屏、知识图谱、三维地图等可视化展示。
- **编辑器方向**：解决编辑器多并发协同、多端状态管理，熟悉 DOM 原生操作。
- **小程序方向**：熟悉微信、支付宝、百度等各种小程序，掌握开发框架如Taro 和 Uniapp。
- **音视频方向**：了解音视频、流媒体基本知识，参与 Web 或小程序音视频 SDK 开发、音视频播放器开发。这是一个偏底层且跨端的技术方向。
- **Web 全栈方向**：既懂前端也会一门后端语言（如 Node.js、Java），能够搭建服务、写 SQL 数据库并进行数据调优。
- **跨端方向**：主要在一套代码多端运行有丰富经验。使用 RN、Flutter、Weex 等进行 Web、iOS、Android、IoT 设备的跨端开发。
- **工程化方向**：占比最大的一个方向，要求从 0 到 1 搭建前端项目、SEO、性能优化、项目运维部署的实战能力。研发效能提升。

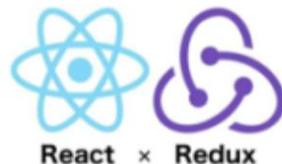
前端相关技术栈、前端工程化

技术栈

- 前端框架和库：使用前端框架包括 React、Vue 和 Angular 加速开发过程。
- 包管理器：使用工具如 npm 或 Yarn 来管理你的项目依赖项。
- 构建工具：Webpack、Parcel 和 Rollup 等工具可以帮助打包和优化前端代码。



Vue 3 全家桶
(eg: vue + vite + ts + pinia)



React 全家桶
(eg: react+redux+react-router+less+es6+webpack)

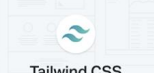
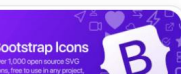
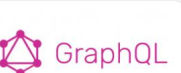
前端工程化是将软件工程的方法和思想应用到前端开发中的过程

- 旨在提高开发效率、减少不必要的重复工作，并提高前端应用的质量。
- 技术选型：选择合适的前端库和框架，例如 React、Vue 等。
- 构建优化：使用构建工具（如Vite、Webpack）对代码进行压缩、合并、转换等操作，以减小文件体积并提高加载速度。
- 模块化开发：采用模块化的方式组织代码，例如 ES6 Module、CommonJS，以便更好地管理和维护。
- 组件化开发与资源管理：将应用拆分成独立的组件，统一管理资源文件，提高代码复用性。
- 规范化：遵循一致的代码风格、命名规范、目录结构等，以便团队协作和维护。
- 自动化测试：使用自动化测试工具对代码进行单元测试、集成测试等，确保代码质量。
- 部署和监控：自动化部署到生产环境，并设置监控系统以实时追踪应用性能。
- 性能优化：优化页面加载速度、减少资源请求，提高用户体验。



前端学习资源列表

资源链接: <https://www.bootcss.com/>

 React React 起源于 Facebook 的内部项目，是一个用于构建用户界面的 JavaScript 库。	 Vue.js Vue 是一套用于构建用户界面的渐进式框架。与其它大型框架不同的是，Vue 被设计为可以自底向上逐层应用。Vue 的核心库只关	 TypeScript 中文手册 TypeScript 是由微软开源的编程语言。它是 JavaScript 的一个超集，而且本质上向这个语言添加了可选的静态类型和基于类的面向对象	 Bootstrap 免费模板 Bootstrap 免费模板 用我们收集的大量优秀的 Bootstrap 免费模板助力您创建精美的 Bootstrap 网站！	 Preact Preact - 一个只有 3kB 大小的 React 替代品，拥有与 React 相同的 API、组件和虚拟 DOM。	 Gulp gulp.js - 基于流(stream)的自动化构建工具。Grunt 采用配置文件的方式执行任务，而 Gulp 一切都通过代码实现。	 Markdown Markdown 是一种轻量级标记语言，便于人们使用易读易写的纯文本格式编写文档并添加格式元素。Markdown 是 John Gruber 于	 Sass Sass 是一个成熟、稳定、强大的 CSS 扩展语言解析器。
 Babel Babel 是一个 JavaScript 编译器。Babel 通过语法转换器支持最新版本的 JavaScript 语法。	 Tailwind CSS Tailwind CSS 是一个用于快速 UI 开发的实用工具集 CSS 框架。与 Bootstrap、Foundation 不同，Tailwind CSS 没有内置的 UI 组	 Alpine.js Alpine.js 通过很低的成本提供了与 Vue 或 React 这类大型框架相近的响应式和声明式特性。Alpine.js 的语法几乎完全借用自	 Redux Redux 是 JavaScript 状态容器，提供可预测化的状态管理。构建一致化的应用，运行于不同的环境（客户端、服务器、原生应用），	 LESS LESS 为 CSS 赋予了动态语言的特性，如变量、继承、运算、函数。LESS 既可以在客户端上运行（支持 IE 6+、Webkit、Firefox），	 Handlebars Handlebars 是一个书写高效率、语义化的模板引擎，与 Mustache 模板兼容。	 Stylus Stylus - 富于表现力、健壮、功能丰富的 CSS 预处理语言。	 Liquid Jekyll 的模板语言 Liquid - 最流行的模板语言。Jekyll、Github Pages 都在用。
 MobX MobX 是一个简单、可扩展的状态管理工具库	 Rollup Rollup 是一个 JavaScript 模块打包工具，可以将小块代码编译成大块复杂的代码。Rollup 对 JavaScript 代码模块使用新的	 Lodash Lodash 是一个具有一致接口、模块化、高性能等特性的 JavaScript 工具库。比相同功能的 Underscore.js 使用更广泛。	 Bootstrap Icons Bootstrap Icons 的设计初衷是与 Bootstrap 组件配合使用。Bootstrap Icons 全部是 SVG 文件，因此能够轻松快捷地进行缩	 EJS EJS 是一套简单的模板语言，帮你利用普通的 JavaScript 代码生成 HTML 页面。EJS 没有再建一套迭代和控制流语法，有的只是普	 Underscore.js Underscore.js 是一个 JavaScript 工具库，它提供了一整套函数式编程的实用功能，弥补了 jQuery 没有实现的功能，同时又是	 Nunjucks Nunjucks 是 JavaScript 专用的功能丰富、强大的模板引擎。	 Zepto.js Zepto.js 是一个轻量级、兼容 jQuery 的 JavaScript 工具库。
 Preact Preact - 一个只有 3kB 大小的	 gulp.js gulp.js - 基于流(stream)的自动化	 Markdown Markdown 是一种轻量级标记语	 Sass Sass 是一个成熟、稳定、强大的	 fastify Fastify，快速并且低开销的 web 框架，专为 Node.js 平台量身打造	 GraphQL GraphQL 既是一种用于 API 的查询语言也是一个满足你数据查询的	 Clipboard.js Clipboard.js 以更现代的方式将文本复制到剪贴板	 Browsersync Browsersync 浏览器同步测试工具，很容易与 Web 平台、构建工

前端开发学习

关注行业趋势

- 行业会议：GMTC全球大前端技术大会、Qcon全球软件开发大会、Web前端开发大会、VueConf、CSSConf等
- 前端公众号：前端之巅、前端早读课等
- 前端趋势报告：StateOfJs、GitHub年度报告各大公司趋势预测等

实践中边开发边学

- 学习资源：框架 / 库官方文档、MDN、W3Schools，掘金社区、SegmentFault 社区、各大开发者社区等
- 掌握调试工具使用：Chrome 调试工具等
- 从开源库中学习：使用三方库，源码学习、架构解析文章学习、参与开源贡献
- 擅用搜索引擎：利用搜索引擎查找答案

夯实基础知识

- 基础学科知识：计算机网络、数据结构与算法、软件工程、编译原理
- HTML/CSS/JS：最基础的核心，红宝书，你不知道的JS……
- 浏览器渲染原理：理解浏览器渲染机制

构建自己的知识图谱

使用丰富优秀的前端开源库

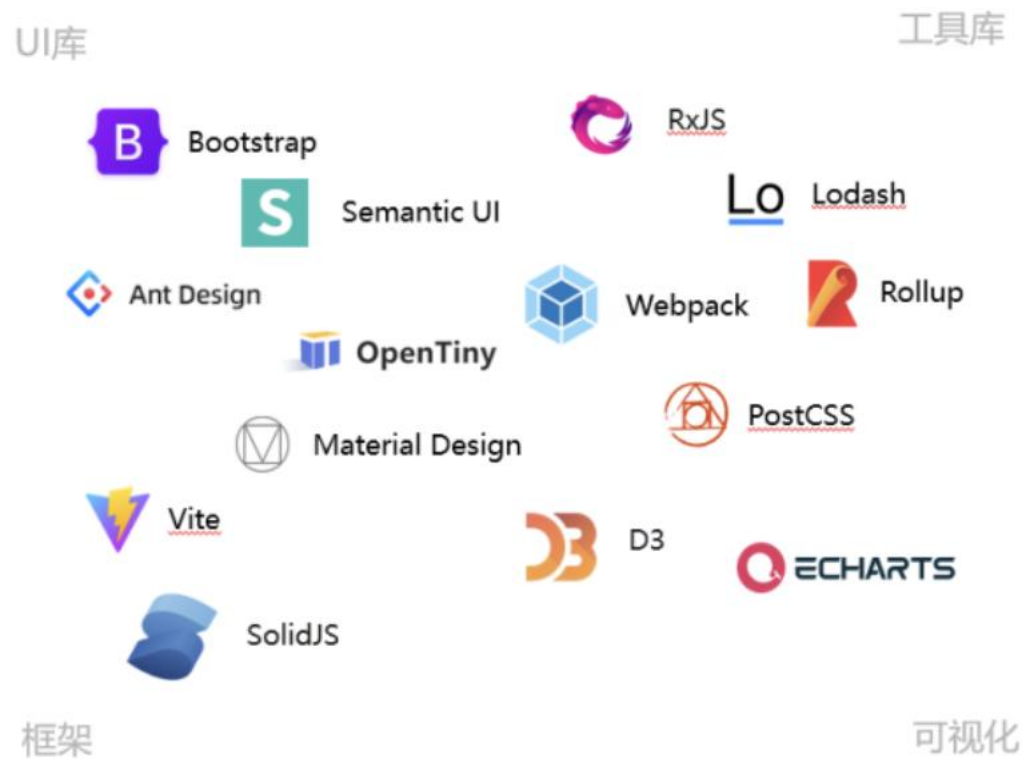
为什么使用开源？

现实：大型软件复杂，全部软件从头实现不现实

使用开源技术进行 Web 开发的好处：

- **降低成本**：商用软件收费较高，自行开发投入人力成本较高；
- **缩短开发时间**：自行开发时长非常长，需要投入大量的人力成本和时间成本；
- **数据保障及安全**：开源代码在应用程序开发过程中发现的安全漏洞通常会立即由社区报告并及时修复；
- **软件质量**：源码公开，且被更多的用户使用和验证，软件质量更可靠；
- **大量工具**：丰富度很高，涉及非常多专业领域相关内容处理；
- **更好的灵活度**：源码公开，用户可以修改它来满足自己的需求；

优秀的前端开源库：



该如何入手学习?

学习资料繁多，技术栈谁好谁坏众说纷纭，该怎么入手?

- 从构建一个简单的应用开发着手
- 选热门的技术栈（资料全 + 讨论多 + 问题解决方案多）
- 从完成 UI 界面开始



+

热门技术栈



实战：使用框架与 UI 库快速搭建 Web 界面

简单实战：

准备：Node.js, Visual Studio Code



配置：npm registry（二选一即可）

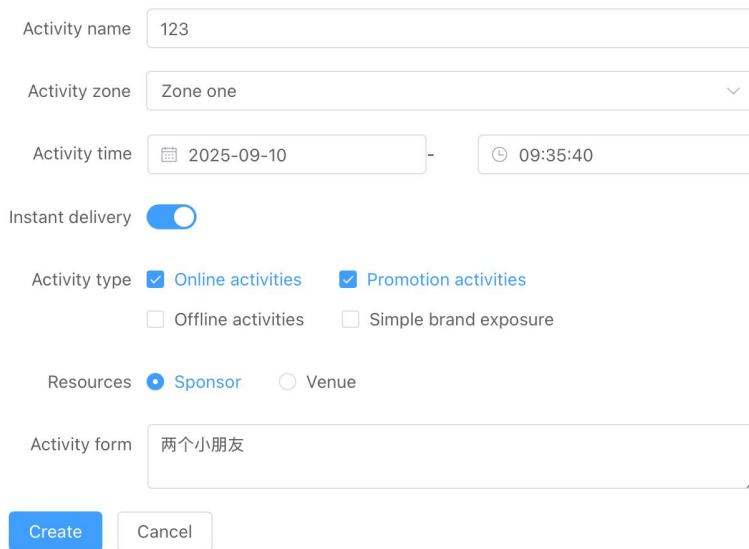
- 淘宝镜像 <https://registry.npmmirror.com>
- 华为云镜像
<https://mirrors.huaweicloud.com/repository/npm/>
- npm config set registry <https://registry.npmmirror.com>

使用 Vite 脚手架，快速生成 Vue 项目，阅读文档

- 快速上手 | Vue.js (vuejs.org)
- 初始化项目 - 安装 volar 扩展 - 启动项目（非必须）

使用 Element Plus UI 组件库，快速实现一个页面

- 任务：实现表单页面

A form interface for creating an activity. It includes fields for Activity name (123), Activity zone (Zone one), Activity time (2025-09-10, 09:35:40), Instant delivery (toggle on), Activity type (Online activities, Promotion activities), Resources (Sponsor, Venue), and Activity form (两个小朋友). There are Create and Cancel buttons at the bottom.

查看说明文档 - 安装组件

选用组件 - 查看 demo - 搭建页面

开源软件使用注意事项

技术选型

- 项目规模、已有的技术栈、开发友好、上手难度

功能匹配度

- 评估功能是否满足需求

使用量 / 热度评估

- NPM 下载热度、GitHub 仓库 Star、版本更新频率、Issue 讨论

许可证

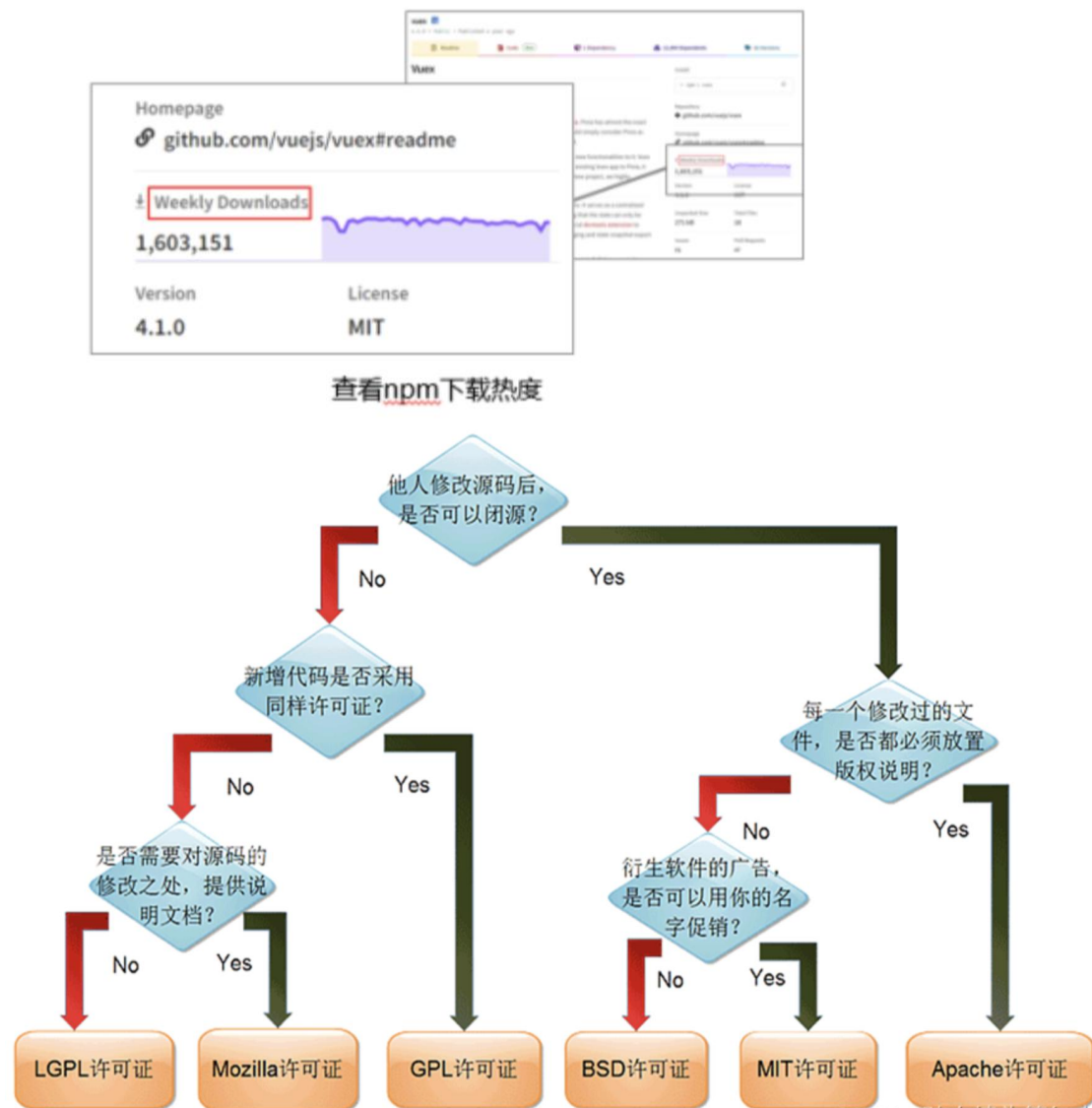
- MIT、GPL、Apache 等，了解项目用途以及未来是否会受限

社区支持

- 社区活跃度，问题讨论热度，有没有官方维护人员答复

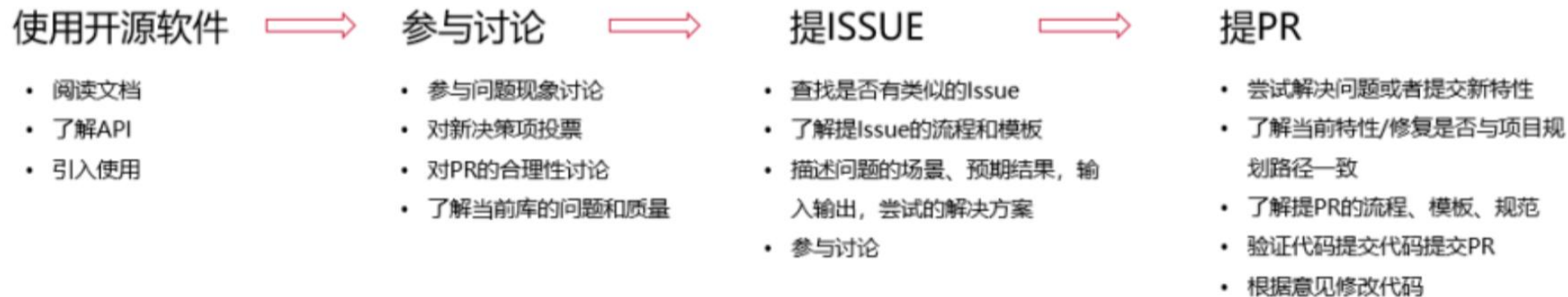
安全漏洞信息

- npm audit / 安全漏洞扫描工具



问题反馈与代码贡献 —— 共建开源生态

使用开源软件并参与软件开发



参与开源贡献的好处：

- 抓住机会、提高技能水平
- 提高软件 / 应用的代码和文档水平
- 接触想法相似的朋友、建立人脉和社区
- 理解应用开发和维护的流程
- 从 Pull Request 反馈中学习
- 学习如何管理自己的开源项目

开源迷思：

- 想法：我不了解代码，开源不适合我。
- 事实：开源不仅仅是编写代码！你可以通过改善文档、测试、创建媒体、内容等各种方式参与进来。所以不要让不具备编码技能成为你参与开源项目的绊脚石。
- 想法：我知道怎么写代码，但是我对开源项目使用的技术不熟悉，我不能贡献。
- 事实：相反，这是学习新东西绝佳的机会！开源生态系统提供足够的时间和耐心让你学习和参与贡献。
- 想法：开源软件总是免费的。
- 事实：大多数开源项目是免费的，但不是所有的都是。这是由项目的许可证所决定的。有一些许可证要求必须让使用和分发免费。需要格外注意项目许可证信息，了解 OSS 到底免费到什么程度。

课程目标

- 掌握前端的核心知识，包括 HTML、CSS、JavaScript;
- 学习 Vue 框架 的使用方法;
- 了解并使用常见的 组件库 和 可视化库;
- 熟悉 Git 工具，进行团队协作开发;
- 能够拉取他人的前端项目，学会运行、使用，甚至进行二次开发。